# Sorted.ai proof of concept

Technical brief for a development house to quote and build

Live product: **https://sorted.trilogybpo.com**. Repository: https://github.com/thegreatcornolio/sorted, branch main. 14 November 2026.

This brief is for a quote and a proof of concept. It describes the live product and the voice path still to add. It sets no price and no service level.

## 1. Purpose and POC outcome

Sorted.ai scores customer-service calls and shows the scores on a live pulse. The product is the live site. A scripted demo already exists on the same site. It is a separate walkthrough, and it is a reference only.

The live scorer accepts text. Audio is not transcribed. This proof of concept adds a place to drop call audio, transcribes it with the customer's existing Deepgram license, and then scores that transcript the same way as a JSON transcript dropped today.

The customer will supply the Deepgram key later. Add a container setting for it. Leave the value out of the repository and out of the static site.

Done means a demonstration can show all of the following.

- An audio file in the voice folder is transcribed. When the file has two tracks, the agent and the caller remain separate speakers. That transcript is then scored.
- A JSON transcript in the transcript folder is scored, with no transcription step.
- The live pages show predicted NPS, CSAT, and Vulnerable Customers.
- A top detractor opens through to that call.
- Vulnerable Customers is its own page. Each row is the person, the reason they were listed, and a suggested next step.
- A failure leaves the source file in its incoming folder and writes an error JSON. A file that has already been scored is left alone until it is dropped into the incoming folder again.

The house delivers the build. Craig (Trilogy BPO) rebuilds the Azure container himself. The work stops at the handover. It does not include a deployment, and it does not include changes to the Azure resources in section 3.

## 2. Users and the live pages they use

People use the system in two ways.

**Dropping files.** A person or an upstream system puts files into Azure Blob storage. The public site has no upload form. Transcripts go into the container incomingtranscripts. This proof of concept adds a container for call audio.

**Reading the pulse.** People open the live site. The browser does not score files. It calls GET /api/pulse and draws the pages. The live menu is Live, Interactions, and Vulnerable Customers. Demo is a separate link, off that menu.

| Page | Route | What it is for |
|---|---|---|
| Live | / and /live | The same overview. NPS dial from −100 to +100, CSAT dial from 1 to 5, sentiment mix, the top five detractor reasons, and one evidence quote. Clear results and Clear transcripts sit at the bottom. |
| Interactions | /live/interactions | Scored files, newest first. Opened from a top reason, the list is the detractors for that reason. Choosing one opens that call. |
| Vulnerable Customers | /live/vulnerable | Its own page, not a tab on Interactions. Each row is the person, why they were listed, and a suggested next step. The NPS and CSAT dials stay on the page. |

A top detractor reason links to /live/interactions filtered to that reason. Opening one call shows the file name, the NPS band, CSAT, the reason, and the quote. The evidence quote on the overview opens the call it was taken from. The unfiltered interactions list shows every scored file and does not open a call.

The pages draw immediately, show a short waiting line, and stop the pulse request after about 4 seconds with a retry. They refresh about every 30 seconds.

The page title is Vulnerable Customers. The text on the page describes the callers as medical inbound.

These routes are the scripted demo. They already exist and they are out of scope: /demo, /demo/interactions, and /scoring. /interactions and /nadclinic open the same scripted queue. A route on the custom domain returns 404 until that route's HTML is committed on main. This proof of concept needs no new route.

## 3. Current MVP

This is what already runs. Keep it. Replace a part only when the replacement still does what this brief describes, including the pulse JSON in section 6. The pages, the scorer, and the transcript drop are the current product. Voice is the path to add.

### Site

https://sorted.trilogybpo.com is a Next.js static export. The domain serves the HTML committed on main. The GitHub Actions Pages artifact is not what the domain serves. A page change reaches the domain when the exported HTML is committed at the repository root (index.html, live/, and the rest of the export). The CNAME file is sorted.trilogybpo.com.

The site build does not include the scorer. The browser reads baseUrl from public/sentiment-pulse-config.json and requests GET /api/pulse on this host:
`https://sorted-pulse.thankfulflower-819372b8.westus3.azurecontainerapps.io`

The response allows only the origin https://sorted.trilogybpo.com. Opened on localhost, the pages draw and then report that the scores did not load. Leave that public origin as it is. For a local demonstration, run the container locally and allow the local origin on that local process only.

### Container

Container app sorted-pulse. West US 3. Resource group sorted-pulse. Registry sortedpulse. Image sortedpulse.azurecr.io/sorted-transcripts:latest.

The process is Node 22, command node dist/container.js, port 8080. It polls every 15 seconds and serves the pulse in the same process. The image contains no secrets. Building the image does not switch the running revision. The site keeps using the previous revision until a new revision of sorted-pulse is created. Craig does that. Build from the repository root with functions/Dockerfile. The image compiles functions/ and does not build the site.

Local run uses Node 22. For the site, npm install and npm run dev. For the scorer, from functions/, with the container settings in the environment: npm install, then npm start (port 8080). npm run once scores whatever is already in incomingtranscripts and exits. npm run check at the repository root checks the site. npm run check inside functions/ runs the scorer tests.

## Storage

Storage account trilogybpo. The scorer does not create the blob containers. They have to exist before the first poll. It creates table SentimentPulse if that table is missing. It does not write Azure SQL.

| Place | Role |
|---|---|
| incomingtranscripts | Drop transcripts here. |
| processedtranscripts | A scored file moves here, under the same blob name. |
| processedtranscripts/results/ | One result JSON per file, and one error JSON when scoring fails. |
| SentimentPulse | One table record. GET /api/pulse reads this record and does not list blobs or call the model. |

## Scoring model

One Azure OpenAI chat completion per file. Resource sorted-openai. Deployment sorted-score. Model gpt-4o. Endpoint https://sorted-openai.openai.azure.com/. API version 2024-12-01-preview. The deployment must accept chat completions with response_format json_schema and strict true. That one completion returns NPS, CSAT, and the vulnerable-customer fields.

Keyword rules score the demo only. They do not score the live pulse.

Container settings. Names only. The values stay on the container app.

| Setting | What it identifies |
|---|---|
| SORTED_STORAGE_CONNECTION_STRING | Storage account trilogybpo. The connection string itself is not in this brief. |
| AZURE_OPENAI_ENDPOINT | https://sorted-openai.openai.azure.com/ with no key in the URL. |
| AZURE_OPENAI_API_KEY | A key for resource sorted-openai. Not stored in the repository. |
| AZURE_OPENAI_DEPLOYMENT | sorted-score |
| AZURE_OPENAI_API_VERSION | 2024-12-01-preview |

Add one further container setting for the Deepgram key when the voice path is built. The customer supplies the value. That key is not in the repository today.

## Clearing the pulse

Both buttons sit at the bottom of the live overview, in one panel.

**Clear results** is POST /api/clear-results. It deletes the score files results/<file>.json and writes an empty pulse. Moved transcripts and any results/<file>.error.json stay in processedtranscripts.

**Clear transcripts** is POST /api/clear-transcripts. The page asks for confirmation. It deletes every blob in processedtranscripts, including moved transcripts, result JSON, and error JSON, and writes an empty pulse.

Neither request touches incomingtranscripts, and neither deletes a container. The next poll does not refill the table from result JSON that has been deleted. A written empty pulse is not copied back from the blobs.

# 4. Ingest: voice and transcripts

## Transcripts

Drop a file in incomingtranscripts. The next poll scores it. Parsing lives in functions/src/score.ts.

A file must be text, non-empty, and at most 2 MB, and it must not contain a null byte. Plain text is accepted, including lines that start with customer:, agent:, advisor:, or caller:. JSON is accepted in any of these shapes:

- A lines array of objects with text and an optional speaker.
- A text string or a transcript string.
- A speech-recognition object that already has text, together with words, audio_duration, confidence, and redacted_pii_policies.

Voice transcription should hand the scorer a lines array, or plain text with speaker labels. Rely on that shape. A raw Deepgram payload is not the speech-recognition shortcut unless it carries the fields above.

company, or a company: line, is stored and becomes the person's name on Vulnerable Customers. A survey number already in the file, nps or score, a whole number from 0 to 10, is stored as nps. Predicted NPS is the estimate. The survey number is not the figure on the pulse, and GET /api/pulse does not return nps. Speech-recognition confidence is not a score.

On success the blob moves to processedtranscripts under the same name, the score is written to processedtranscripts/results/<name>.json, any previous results/<name>.error.json is removed, and the score is added to SentimentPulse. A later score for the same file name replaces that file's row.

On failure the blob stays in incomingtranscripts. The next poll tries it again. The error is written to processedtranscripts/results/<name>.error.json with file, error, and failedAt.

A file that has moved to processedtranscripts is not scored on later polls. To score it again, drop it into incomingtranscripts.

Audio does not survive this path. A null byte fails as “File is not text.” Call recordings need their own container. This is a transcript the scorer already accepts, and it is the shape transcription should produce:

```
{
  "company": "Example Clinic",
  "lines": [
    { "speaker": "agent", "text": "Thanks for calling. How can I help?" },
    { "speaker": "caller", "text": "I can't afford the payment this month." }
  ]
}
```

## Voice

Add a blob container named incomingaudio on storage account trilogybpo. It does not exist yet. Create it before the first poll, as the transcript containers already have to exist. Keep audio out of incomingtranscripts.

Poll incomingaudio on the same 15 second cycle. For each new file:

1. Send the audio to Deepgram with the customer's existing license. The customer supplies the key later. The key is a container setting.
2. When the file has two tracks, transcribe the tracks separately and keep the agent and the caller as separate speakers. Label them agent and caller (customer is also accepted). Do not merge the two tracks into one speaker. If the recorder puts the agent on a fixed track, use that order. Ask Trilogy for the track order before building if it is not already known.
3. A single-track file is still transcribed and scored. It may have no speaker labels.

4. Pass the transcript to the existing scorer in the shape above, with the speaker labels still on the lines, so the model can see who spoke.
5. After the score is stored, move the audio out of incomingaudio so a later poll does not transcribe it again. Keep the transcript text with the processed files so the score can be checked against what was said.
6. If transcription or scoring fails, leave the audio in incomingaudio and write processedtranscripts/results/<name>.error.json. The next poll retries. A later success removes that error file.

Name the result after the audio file, the same way a transcript result is named: results/<name>.json. Use distinct file names for audio and for JSON transcripts. The pulse keeps one row per file name.

State the accepted audio types in the quote. The acceptance recording is a two-channel WAV, and the proof of concept also accepts a common compressed recording such as MP3. State the maximum audio size in the quote. The 2 MB cap is for text, not for audio.

Audio that has already been moved out of incomingaudio is not transcribed again until the file is dropped there again.

## 5. Scoring and the vulnerable rule

Score every transcript, including one that came from Deepgram, with one chat completion on deployment sorted-score. The request is limited to about 45 seconds. A timeout is a scoring failure. Text longer than 100,000 characters is clipped before the request: the opening, a note that the middle was omitted, and the ending. The instruction is to judge the whole call and weigh the ending.

The completion returns strict JSON:

- predicted_nps. A whole number from 0 to 10. An estimate, not a survey score.
- nps_category. promoter, passive, or detractor.
- sentiment_start and sentiment_end. Each is positive, mixed, or negative.
- resolved. True only when the caller's request was completed, including a booked and paid appointment.
- csat. A whole number from 1 to 5.
- primary_reason. One short sentence.
- evidence. One sentence copied from the transcript.
- summary. One or two sentences on the outcome.
- vulnerable. True or false.
- vulnerability_reason and next_step. Filled when vulnerable is true. Empty when it is false.

A booked, paid close is not a detractor. A caller who books and pays after disliking the price or the wait is a promoter at 9 with CSAT 4. A clearly pleased close can be 10 with CSAT 5. An unresolved complaint, a refusal, a cancellation, or an angry ending is a detractor: predicted NPS 0 to 6 and CSAT 1 or 2. A genuinely undecided caller is passive: predicted NPS 7 or 8 and CSAT 3.

The scorer checks these pairs and rejects a result that breaks them. That rejection is a scoring failure.

- Promoter: predicted NPS 9–10, and CSAT 4 or 5.
- Passive: predicted NPS 7–8, and CSAT 3.
- Detractor: predicted NPS 0–6, and CSAT 1 or 2.

sentiment_end is the sentiment stored for the pulse. predicted_nps is stored as predictedNps. CSAT is also stored as a band: positive for 4–5, neutral for 3, negative for 1–2.

Evidence has to be a verbatim sentence from the transcript, at least four words, at most 280 characters. A fragment such as “Yes.” is rejected. The sentence has to occur in the transcript. The scorer rejects a quote that does not.

The instruction text and the JSON schema are in the repository at functions/src/openai.ts. Keep that schema on the live scorer.

### Company NPS and CSAT

Predicted NPS is an estimate, not a survey score. Promoters are 9–10. Passives are 7–8. Detractors are 0–6.

Company NPS is the percentage of promoters minus the percentage of detractors. Passives stay in the total and are left out of the subtraction.

```
NPS = round( (promoters − detractors) / total × 100 )
```

The dial runs from −100 to +100. With no scored calls it stays empty. One promoter and two passives is a total of 3, so the NPS is 33. CSAT on the dial is the average of the per-call 1–5 estimates, to one decimal place.

### Vulnerable customers

vulnerable is true only when the caller has trouble paying, is late on a payment, or describes a change in circumstances that means they cannot pay. Saying they cannot afford the service, including “I can't afford that”, is trouble paying.

Ordinary price dislike is a high or unfair price when the caller can still pay. That call is not listed as vulnerable.

When vulnerable is false, vulnerability_reason and next_step are empty. When it is true, the reason is one short sentence on why they were listed, and the next step is one short suggestion for the team. Both are required. The page drops a vulnerable row that is missing either one.

The live scorer also corrects a miss. If the model marks the call as not vulnerable, and the transcript, the reason, or the evidence shows the caller cannot afford, cannot pay, or is late on payments, the stored score is vulnerable and both strings are filled. A change in circumstances that means they cannot pay is part of the instruction to the model, and that call still needs a reason and a next step.

On the page, the person is the file's company when that field is present. Otherwise it is the file name without its extension.

## 6. Data the page needs

The three live pages read GET /api/pulse only. They do not read blobs. The body is the aggregate of the one SentimentPulse record, as JSON, with Cache-Control no-store. The CORS origin is https://sorted.trilogybpo.com.

| Field | What the page uses |
|---|---|
| total | Number of scored calls. |
| mix | positive, neutral, and negative. Whole-number percentages of sentiment_end. Mixed is counted as neutral. The three shares sum to 100 when total is not zero. |
| promoters | Count of predicted NPS 9–10. |
| detractors | Count of predicted NPS 0–6. |
| csat | Average of the 1–5 estimates, one decimal, or null when there are no scores. |
| reasons | Detractor reasons only: label and count, highest count first. The page shows the top five. |
| evidence | quote and file, or null. The quote links to that call. |
| movement | One point per scored file, oldest first: file, scoredAt, positive, negative. Interactions lists these newest first. The percentages are the running mix after that file. |
| calls | Every detractor: file, scoredAt, npsBand set to detractor, csat, reason, and quote. The top-reason links need this list. Without it, a reason cannot open the call. |

| Field | What the page uses |
|---|---|
| vulnerable | person, file, vulnerable true, vulnerability_reason, next_step, and scoredAt. A row missing the person, the file, the reason, or the next step is not shown. |

The page computes passives as total minus promoters minus detractors, and company NPS as in section 5. It does not read a channel. Live files have no channel field, and the page does not add one.

POST /api/clear-results and POST /api/clear-transcripts return the same JSON shape, empty, after they write the empty pulse.

When processedtranscripts has no results/*.json, the next poll rewrites SentimentPulse as an empty pulse. When result JSON is present and the table record has never been written, that poll copies the results in. A pulse that has been written empty is not refilled from the blobs.

The packed table record has to stay within 900,000 characters. Above that, the pulse write fails. A short proof-of-concept batch sits inside that cap. Say so in the quote if the batch will be large.

## 7. Suggested build scope, out of scope, and acceptance checks

### What to build

1. Container incomingaudio, a poll, Deepgram transcription with the customer's license, separate speakers when the file has two tracks, then the existing score path.
2. A container setting for the Deepgram key. The customer supplies the value later.
3. The current transcript drop, scorer, pulse API, clear actions, and three live pages, kept as they are. Change one only where the voice path needs the change, or where an acceptance check fails.
4. Short notes Craig can follow to build image sorted-transcripts and create a new revision of container app sorted-pulse. He runs that step. The house does not deploy, and does not change the resource group, the registry, the storage account, or the OpenAI resource.

A replacement for the container process is acceptable when it still listens on port 8080, polls about every 15 seconds, and serves GET /api/pulse plus the two clear routes with the JSON in section 6.

No new public page is required. A new route returns 404 on the custom domain until its HTML is committed on main.

### Out of scope

- The scripted demo at /demo, /demo/interactions, and /scoring, including the aliases /interactions and /nadclinic. Keyword scoring and the demo's next-best-action stay on that walkthrough. They must stay off the live pulse. The demo is a reference that already exists.
- Email and WhatsApp as live channels. The demo uses them. Live files have no channel.
- Collecting a survey NPS. Predicted NPS is the estimate. A survey number already written in a file stays off the pulse.
- A new Azure subscription, region, OpenAI resource, or storage account.
- Deploying or revising sorted-pulse. Craig (Trilogy BPO) does that.
- A price, a retention period, or a service level. This brief sets none of those.
- The local upload page from npm run upload in functions/, on 127.0.0.1:8787. It is not on the public site. It only puts text files into incomingtranscripts.

## Acceptance checks

Demonstrate these on a local run of the container and the site, or on a build Craig has placed on sorted-pulse. Run the voice checks after the customer has supplied the Deepgram key. The scripted demo is not the demonstration.

1. Drop a JSON transcript into incomingtranscripts. On a following poll it moves to processedtranscripts, results/<name>.json is written, and the live overview shows NPS and CSAT.
2. Score a pleased close, an unresolved or angry ending, and an undecided caller. Stored bands follow the number: 9–10 promoter, 7–8 passive, 0–6 detractor. CSAT is 4–5, 3, and 1–2 for those bands. Company NPS on the page matches the formula in section 5. A booked, paid close is not stored as a detractor.
3. Drop a transcript where the caller cannot afford it, cannot pay, is late on payments, or describes a change in circumstances that means they cannot pay. That person appears on /live/vulnerable with a reason and a suggested next step. Drop a transcript that only objects to the price, and where they can still pay. That person is absent from the list.
4. On the overview, click a top detractor reason, then open that call. The call shows the file, the reason, the quote, the NPS band, and CSAT. The evidence quote on the overview opens the call it came from.
5. Vulnerable Customers is the route /live/vulnerable and its own item on the live menu. It is not a tab on Interactions.
6. Drop a two-channel WAV into incomingaudio. The stored transcript has the agent and the caller on separate lines, and the call then appears on the pulse the same way as a dropped transcript. A single-track file is transcribed and scored as well.
7. Drop an empty or invalid transcript. It remains in incomingtranscripts. results/<name>.error.json contains file, error, and failedAt. It is not listed as a scored call. A voice file that transcription or scoring rejects stays in incomingaudio, with the same kind of error JSON.
8. A later poll does not score a file that has already left the incoming folder. Dropping that file into the incoming folder again scores it again.
9. Clear results deletes the result JSON and empties the pulse. Moved transcripts, error JSON, and incoming files remain. Clear transcripts, after confirmation, deletes the blobs in processedtranscripts and empties the pulse. Incoming files remain. Both buttons are at the bottom of the live overview.
10. The route /demo still opens the scripted walkthrough. Scores on the live pages come from dropped files.
11. The handover contains no API key, no Deepgram key, and no storage connection string.

## 8. Architecture

Voice is transcribed, then scored on the same path as a dropped transcript. The site reads the pulse. It does not score, and it does not read the blobs.

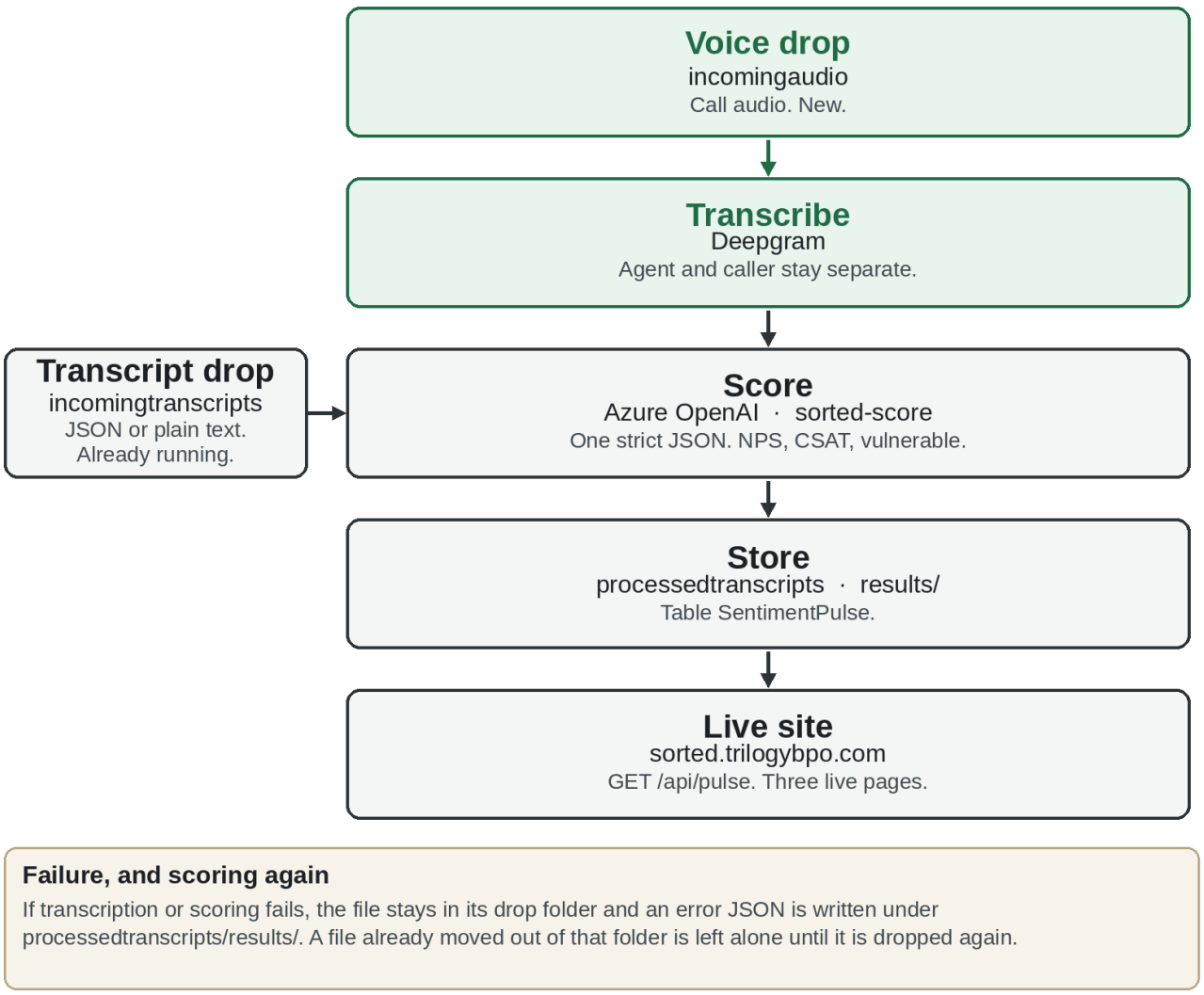



*Figure 1. Drop, transcribe, score, store, and the live site. Green marks the voice path this proof of concept adds. Grey is what already runs. A dropped transcript skips transcription and enters at Score.*